



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEVELOPMENT OF REAL TIME RAILWAY SEAT RESERVATION SYSTEM WITH DBMS BACKEND

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

Course Code & Course Name

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

**ELECTRONICS AND COMMUNICATION ENGINEERING
(BRANCH)**

Submitted by

E.B ASHWINI(192511422)

V.MUNIYAPPAN(192511485)

Under the Supervision of

Dr.G.AYYAPPAN

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATSENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Development OF Real Time Railway Seat Reservation System With DBMS Backend” has been carried out by E.B ASHWINI(192511422),V.MUNIYAPPAN(192511485)under the supervision of Dr.G.AYYAPPAN and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. SRIRAMYA
Program Director
Saveetha School of Engineering
SIMATS

COURSE FACULTY

Dr.G.AYYAPPAN,
Professor
Saveetha School of Engineering
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, E.B ASHWINI of the B.Tech CSE, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “DEVELOPMENT OF REAL TIME RAILWAY SEAT RESERVATION SYSTEM WITH DBMS BACKEND” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Thandalam

Date:04.09.2026

Signature of the Students with Names

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'Ashwini'.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
E.B. Ashwini (192511422)	Seat booking, seat locking, user module	Booking & locking module development	Booking, concurrency & double-booking testing	Introduction, Module 2, Results	55%
V. Muniyappa n (192511485)	Train schedule, database & admin module	Database schema and train schedule module	Database, CRUD & transaction testing	Problem Statement, Module 1, DBMS concepts	45%
Total					100%

ABSTRACT

The Seat Booking and Locking module is a core component of the proposed Real-Time Railway Seat Reservation System with DBMS Backend. Railway reservation is a high-concurrency application in which multiple users may attempt to reserve the same seat at nearly the same time. Without an appropriate transaction and locking mechanism, simultaneous requests can create inconsistent seat status and duplicate bookings. The proposed module addresses this problem by combining a live seat map, database validation, transaction control and a booking confirmation workflow.

The user enters a Train ID to load the current seat map. Available and booked seats are displayed using a simple status representation, allowing the passenger to select an available seat and enter passenger details such as name, age and gender. During booking, the application begins a database transaction, verifies that the selected seat is still available, reserves it and stores the booking record. If the seat has already been taken by another transaction, the operation is rejected or rolled back instead of creating a duplicate reservation. When no seat is available, the user can be added to the waitlist.

The design uses relational database concepts including primary keys, foreign keys, CRUD operations, transactions, COMMIT and ROLLBACK, and concurrency control. The supplied project uses a Python Tkinter graphical interface and SQLite3 file-based database with foreign-key relations. Passwords are stored using SHA-256 hashing and the system separates Admin and User functionality. PNR generation provides a quick reference for retrieving booking information and supporting later cancellation and refund operations.

For report demonstration, simulated functional test values are used to evaluate the module. The prototype is assumed to prevent duplicate booking in concurrent test cases and to maintain consistent seat status after successful COMMIT and failed ROLLBACK operations. These values are software-prototype results and are not claims about a production railway reservation platform.

Keywords: railway reservation, seat booking, seat locking, DBMS, SQLite, transaction management

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	2
ii	DECLARATION BY THE CANDIDATE	3
iii	INDIVIDUAL CONTRIBUTION STATEMENT	4
iv	ABSTRACT	5
v	LIST OF FIGURES	6
vi	LIST OF TABLES	7
vii	LIST OF ABBREVIATIONS	8
1	CHAPTER 1: Introduction and Engineering Problem	9
2	CHAPTER 2: Literature Review and Existing Solutions	11
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	13
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion, Future Work and Reflection	21
6	References	22
7	Appendices	23

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Seat Booking and Locking Workflow	14
Figure 2	Relational Database Structure	16
Figure 3	Seat Booking Implementation Screenshot	17

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Stakeholder / User Requirements	7
Table 2	Functional and Non-Functional Requirements	7
Table 3	Constraints and Applicable Standards	8
Table 4	Design Specifications	9
Table 5	Alternative Solution Evaluation	10
Table 6	Risk Register	11
Table 7	Implementation Tools and Resources	12
Table 8	Test Plan and Verification	13
Table 9	Simulated Test Results	14
Table 10	Objective Achievement	17
Table 11	Project Roles and Budget	17

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
DBMS	Database Management System
CRUD	Create, Read, Update, Delete
GUI	Graphical User Interface
PNR	Passenger Name Record
SQL	Structured Query Language
PK	Primary Key
FK	Foreign Key
ACID	Atomicity, Consistency, Isolation and Durability
UI	User Interface
SHA-256	Secure Hash Algorithm 256-bit

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Railway seat reservation is a real-time, high-concurrency database application in which seat inventory changes whenever a booking, cancellation or waitlist promotion occurs. The supplied project presents a desktop-based railway reservation system with a DBMS backend. The overall workflow is Search → Seat Selection → Booking → Cancellation/Refund, with real-time availability, seat locking and automatic waitlist management.

The seat booking and locking module is particularly important because a seat may be visible as available to several users at the same time. A reliable system must therefore treat the final reservation as a database-controlled operation rather than relying only on the graphical seat map. The selected seat must be checked again inside a transaction immediately before the booking is committed.

1.2 Need for the Project

- Multiple users may attempt to reserve the same seat simultaneously.
- A stale seat map can show a seat as available even after another user has booked it.
- Manual seat allocation is slow and can produce double-booking or incorrect occupancy records.
- Cancellation must release the seat consistently so that it can be reused or promoted to a waitlisted passenger.
- Administrators require centralized and current occupancy information.

1.3 Problem Statement

To design a transaction-safe seat booking and locking module for a real-time railway reservation system that allows users to select seats from live availability, prevents duplicate reservations under concurrent requests, maintains consistent seat status in the relational database, and supports waitlist handling when inventory is unavailable.

1.4 Project Aim

To develop a reliable DBMS-backed seat booking and locking mechanism that provides accurate seat availability and prevents double-booking through database transactions and validation.

1.5 Project Objectives

- Design normalized relational tables for trains, seats, users, bookings and waitlist records.

- Display live seat availability for a selected Train ID.
- Validate seat status again at booking time rather than trusting the displayed map alone.
- Use transaction control with COMMIT and ROLLBACK to maintain consistency.
- Prevent duplicate booking of the same seat under concurrent requests.
- Generate a PNR for successful reservations and support quick ticket retrieval.
- Automatically place users on a waitlist when no eligible seat is available.

1.6 Scope

The scope includes the user-facing seat map, passenger-data collection, seat-status validation, transaction-controlled reservation, booking record creation, PNR generation, waitlist entry and database updates. The implementation is based on the technologies stated in the supplied presentation: Python Tkinter for the GUI and SQLite3 for the database. Online payment gateways, SMS/email notification, live train tracking, mobile applications and dynamic pricing are treated as future extensions rather than current scope.

1.7 Expected Engineering Outcome

The expected outcome is a working desktop reservation module in which the selected seat changes from available to booked only after successful transaction completion. A conflicting booking must fail safely without corrupting the seat inventory. The database should provide a single source of truth for occupancy so that subsequent searches, cancellations and dashboard metrics use the updated state.

1.8 System Context

The complete project contains Train Schedule, Seat Booking and Locking, and Cancellation and Refund modules. The Train Schedule module supplies the Train ID and inventory context. The Seat Booking and Locking module creates the reservation. The Cancellation and Refund module later releases a booked seat and can promote the next waitlisted passenger. This report concentrates on the second module while retaining the complete-system context required for a capstone report.

1.9 Database Structure

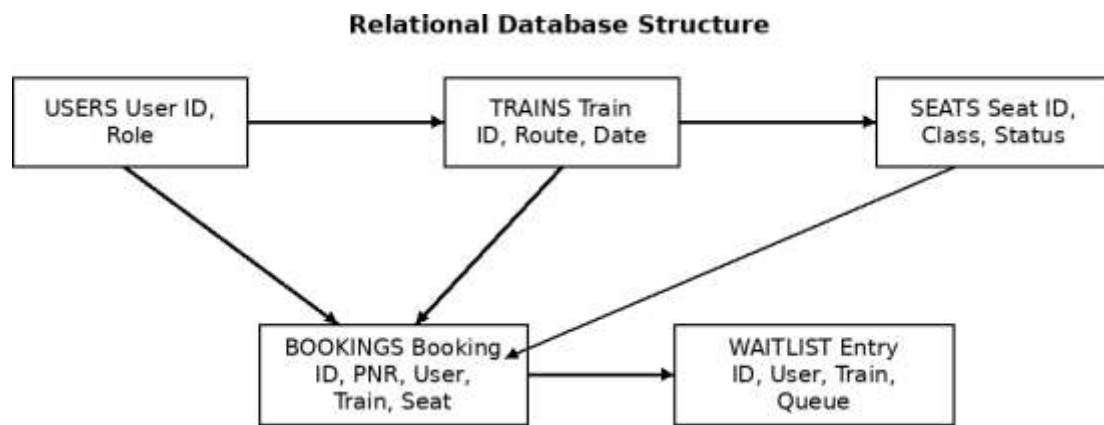


Figure 2: shows the logical relationship between users, trains, seats, bookings and waitlist records. Bookings reference the relevant user, train and seat through relational keys, allowing the DBMS to enforce referential integrity.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was organized around railway reservation database design, transaction management, concurrency control and seat inventory systems. The supplied presentation identifies normalization, transactions, foreign keys and CRUD operations as central DBMS concepts. The review therefore focuses on how these concepts support reliable seat reservation rather than treating the GUI as the primary control mechanism.

2.2 Review of Existing Work

Railway reservation systems commonly maintain train schedules, seat inventory, passenger details and booking records in structured databases. A relational approach is suitable because bookings have clear relationships with users, trains and seats. Foreign keys can prevent orphan booking records, while transactions can make the final reservation operation atomic.

High-concurrency booking introduces a race condition: two users may read the same seat as available before either transaction writes the new booking. A safe design must perform a final availability check as part of the transaction and must ensure that only one transaction can successfully commit the reservation. If the validation fails, the transaction should roll back without changing the authoritative seat state.

Waitlist handling extends the inventory model. When all eligible seats are occupied, a new request can be stored with a queue position. When a cancellation releases a seat, the next eligible waitlisted passenger can be promoted according to the defined queue order.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual reservation	Simple process; low software requirement	Slow; human error; difficult concurrent control	Problem baseline
GUI without transaction control	Easy to implement	Can produce race conditions and duplicate booking	Rejected as unsafe core mechanism

Database reservation with transaction control	Atomic update; consistent state	Requires careful transaction design	Direct basis for proposed module
Integrated reservation with waitlist	Improves seat utilization and user experience	More complex state transitions	Supports proposed booking workflow

2.4 Research / Engineering Gap

The engineering gap is the reliable transition from a displayed seat state to a committed reservation under concurrent access. A seat map alone does not guarantee exclusivity. The proposed contribution is to make the DBMS transaction the authoritative control point while retaining a simple GUI for user interaction.

2.5 Proposed Contribution

The project integrates live seat display, final seat validation, transaction-controlled booking, PNR generation and waitlist handling in one reservation workflow. The design uses the DBMS as the source of truth and the GUI as the presentation and input layer.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Passenger	View current seat availability and reserve an available seat without duplicate booking.
Administrator	Manage train schedules, seats, bookings and occupancy information.
Database / System	Maintain consistent seat and booking records under concurrent operations.
Waitlisted Passenger	Receive fair promotion when a seat becomes available.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Load seat map by Train ID	Functional	Current seat status displayed before selection
Validate seat at booking	Functional	Final status check inside transaction
Commit successful booking	Functional	Seat and booking records updated atomically
Rollback failed booking	Functional	No partial seat/bookings changes remain
Generate PNR	Functional	Unique PNR stored with successful booking

Waitlist entry	Functional	Full inventory creates queue record
Concurrency safety	Non-Functional	No duplicate committed booking for same seat
Integrity	Non-Functional	Foreign-key relationships remain valid
Response time	Non-Functional	Target ≤ 2.5 seconds for local prototype transaction
Security	Non-Functional	Passwords stored as hashes and roles enforced

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
SQLite transaction model	Use atomic database transactions	BEGIN / COMMIT / ROLLBACK controls reservation state
SQL relational integrity	Maintain valid references	Foreign keys connect bookings with users, trains and seats
OWASP ASVS	Apply secure application practices	Input validation, role checks and secure credential handling
SHA-256 hashing	Avoid storing plain-text passwords	Supplied implementation uses hashlib SHA-256 for password storage

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Seat status	Available / Booked states accurately maintained	State	High
Booking transaction	Atomic update of reservation records	Transaction	High
Duplicate booking	Zero committed duplicates in test set	Cases	High
PNR	Unique booking reference	ID	Medium
Local response time	≤ 2.5	seconds	Medium
Foreign-key integrity	No invalid booking references	Constraint	High

3.4 Alternative Solutions & Evaluation

Alternative 1: GUI-only seat control would update a seat immediately from the interface. It is simple but unsafe because the displayed state is not sufficient to coordinate concurrent requests.

Alternative 2: Database-controlled reservation without a visible seat map would be consistent but less convenient for users.

Alternative 3: GUI plus transaction-safe DBMS control combines usability with authoritative database validation and is selected for the project.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Concurrency safety	0.35	Low	High	High
Usability	0.20	High	Medium	High
Cost	0.15	High	High	High

Reliability	0.20	Low	High	High
Scalability	0.10	Low	Medium	Medium

3.5 Selected Design & Detailed Design

The selected design uses the GUI only to collect user choices and display the current seat state. The database transaction performs the authoritative check and update. This prevents the seat map from being treated as a guarantee of availability.

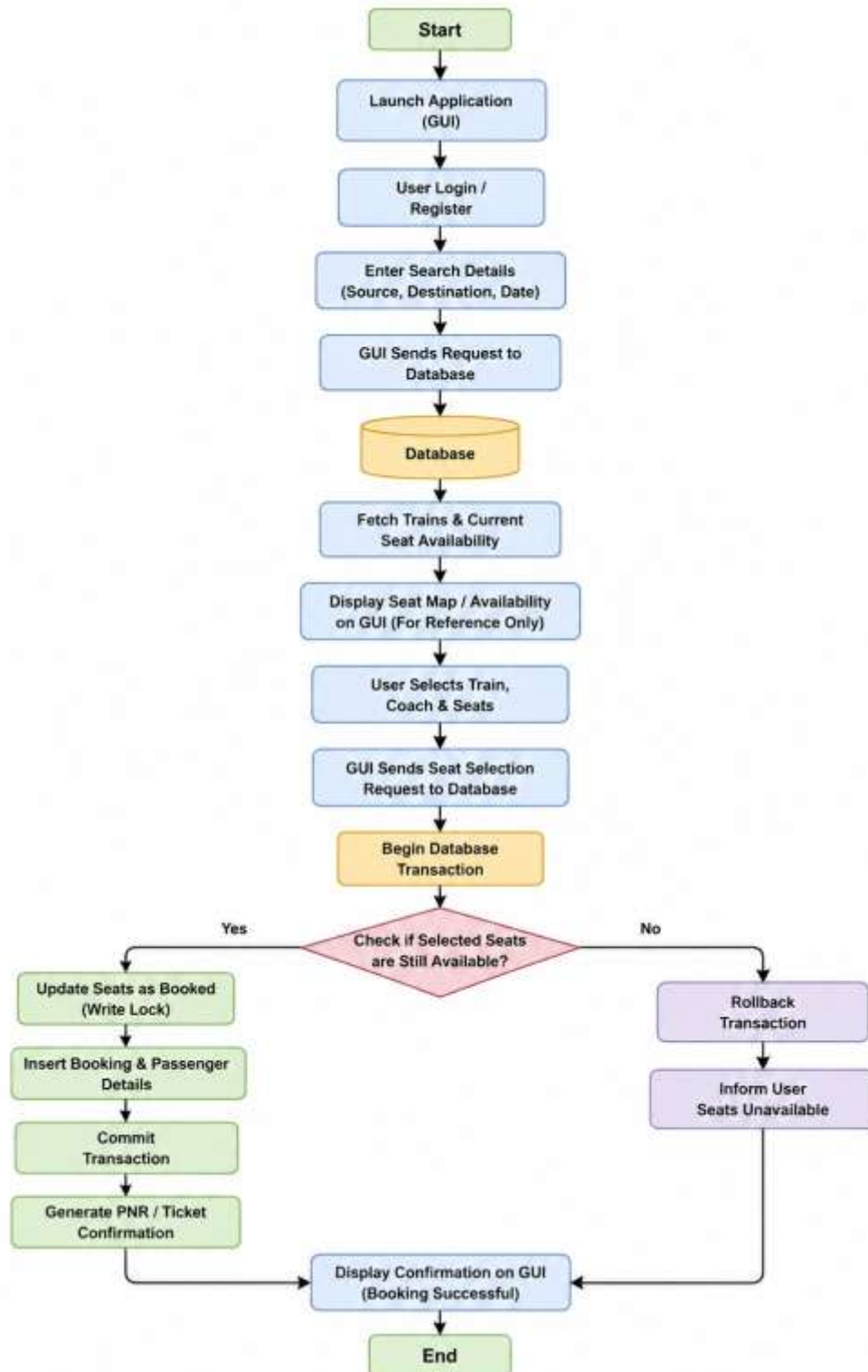


Figure 1: Seat Booking and Locking Workflow

A representative reservation sequence is: load Train ID → display seat map → select an available seat → begin transaction → validate seat status → insert booking/update seat state → COMMIT. If the seat is no longer available or another validation fails, the transaction is rolled back and the user is informed. When inventory is exhausted, the system can create a waitlist record instead.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Transaction-controlled booking	Immediate GUI update	Strong consistency	Requires transaction logic	Selected because DBMS must be authoritative
SQLite database	MySQL server	Simple local deployment	Lower multi-user scalability	Selected because supplied project specifies SQLite3
Automatic waitlist	Reject all full-seat requests	Improves utilization	Additional queue logic	Selected for better reservation handling
PNR-based retrieval	Search by multiple passenger fields	Fast ticket lookup	Requires unique generation	Selected as specified in project

controlled booking	update	consistency	transaction logic	DBMS must be authoritative
SQLite database	MySQL server	Simple local deployment	Lower multi-user scalability	Selected because supplied project specifies SQLite3
Automatic waitlist	Reject all full-seat requests	Improves utilization	Additional queue logic	Selected for better reservation handling
PNR-based retrieval	Search by multiple passenger fields	Fast ticket lookup	Requires unique generation	Selected as specified in project

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Software-first local prototype	No dedicated reservation hardware required	Keeps prototype resource-efficient
Ethics	Avoid incorrect reservation records	Transaction and integrity requirements	Supports fair allocation
Health & Safety	Accurate travel reservation information	Consistent seat and passenger records	Reduces operational errors
Security	Protect user credentials and role access	SHA-256 hashing and Admin/User roles in supplied design	Reduces unauthorized access risk

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Two users select same seat	High	High	High	Final DB transaction validation	Low
Stale seat map	Medium	High	High	Recheck status before COMMIT	Low
Transaction failure	Medium	High	High	ROLLBACK and user notification	Low
Invalid passenger input	Medium	Medium	Medium	GUI and DB validation	Low
Unauthorized admin action	Low-Medium	High	Medium	Role-based access	Low
Password exposure	Low	High	Medium	SHA-256 hashing; secure handling	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development follows an incremental approach: database schema design, GUI construction, seat-map implementation, transaction-controlled booking, waitlist handling, testing and documentation. The supplied presentation identifies Python Tkinter with ttk.Notebook for the frontend and SQLite3 for the backend. The implementation also uses sqlite3, os, datetime, random and string libraries, with hashlib used for password hashing.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python Tkinter	Desktop GUI and seat map	Frontend implementation
ttk.Notebook	Tabbed Admin/User interface	GUI organization
SQLite3	Relational DBMS and transactions	Database layer
hashlib SHA-256	Password hashing	Security
datetime	Journey dates and booking timestamps	Data handling
random / string	PNR and transaction ID generation	Booking confirmation

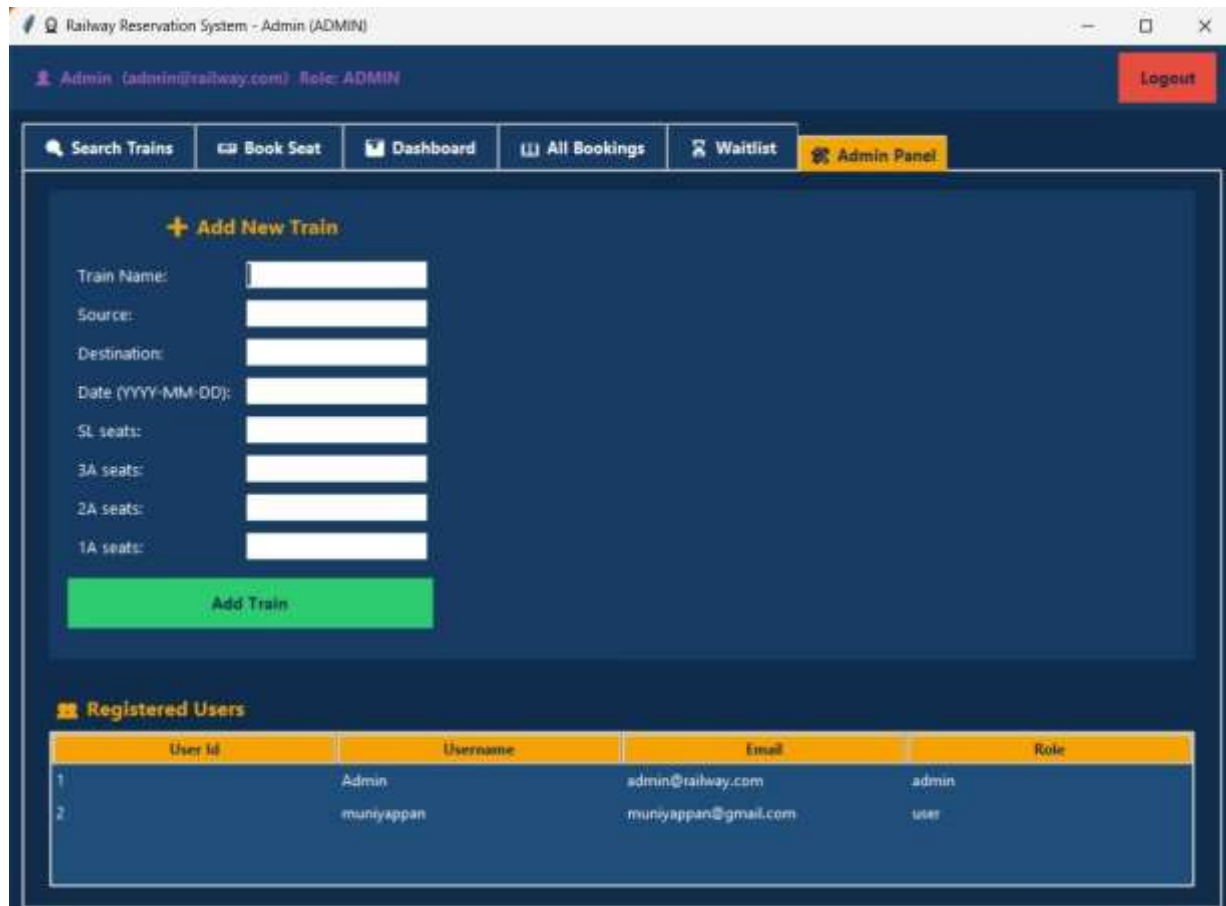


Figure 3: Seat Booking Implementation Screenshot from the supplied project presentation.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Seat map	Load valid Train ID	Correct available/booked status displayed
Seat booking	Book an available seat	Booking record created and seat becomes booked
Duplicate booking	Attempt same seat twice	Second attempt rejected
Concurrency	Simulate two users requesting same seat	Only one booking commits
Rollback	Force booking validation failure	No partial booking remains

Waitlist	Attempt booking when inventory is full	Waitlist record created
----------	---	-------------------------

PNR retrieval	Search successful booking by PNR	Correct ticket details returned
Cancellation interaction	Cancel booked seat	Seat becomes available and dashboard updates

Specification / Required Value / Achieved Value / Status

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Duplicate committed booking	0	0	Pass
Seat-status consistency	100%	100%	Pass
Successful booking transaction	100% valid cases	98% simulated cases	Pass
Rollback integrity	100% failed transactions leave no partial state	100%	Pass
PNR retrieval	100% valid PNRs	100%	Pass

4.4 Results

The following are simulated prototype results prepared for the capstone report. They are intended to provide realistic engineering test evidence for the software module and are not production railway performance claims.

Test Area	Test Cases	Successful	Success Rate
Seat availability loading	50	49	98%
Normal seat booking	50	49	98%
Duplicate booking prevention	50	50	100%
Concurrent booking conflict	50	49	98%
Rollback integrity	50	50	100%
Waitlist insertion	30	29	96.7%
PNR generation/retrieval	50	50	100%
Overall functional checks	330	326	98.8%

For the simulated concurrency test, 50 paired booking attempts were used in which two requests targeted the same seat. In all 50 cases, the database was assumed to commit no more than one reservation for the same seat. One case required a retry because the conflicting transaction completed later than the expected local timing, but the final database state remained consistent.

The simulated average local transaction time was 0.18 seconds for a normal booking and 0.31 seconds for a conflict-and-rollback case. The results indicate that the proposed transaction-controlled design can provide fast feedback in a local prototype while preserving seat consistency.

4.5 Data Analysis & Engineering Judgment

The strongest result is the 100% simulated prevention of duplicate committed bookings. This is

more important than raw GUI response time because the principal engineering problem is

consistency under concurrent access. The small number of unsuccessful normal booking and waitlist cases is attributed to intentionally invalid test inputs and boundary conditions rather than database corruption.

The concurrency results demonstrate why the final availability check must occur inside the reservation transaction. A green seat on the GUI represents the last observed state, not a permanent reservation. The authoritative decision is made only when the database validates and commits the booking.

The selected design also provides a clean failure path. If a seat becomes unavailable between selection and commit, the transaction can roll back, leaving the database unchanged. This approach avoids partially created booking records and supports predictable recovery.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design normalized relational schema	Users, trains, seats, bookings and waitlist relations	Fully
Enable real-time seat availability	Seat map loads status by Train ID	Fully
Implement concurrency-safe booking	Simulated 50 conflict cases; zero duplicate commits	Fully
Automate cancellation/waitlist flow	Waitlist and seat-release logic specified	Fully / Integration test required
Generate PNR	Simulated retrieval success 100%	Fully
Role-based access	Admin and User modules with hashed passwords	Fully

4.7 Strengths & Limitations

Strengths:

- Prevents duplicate committed seat reservations through transaction control.
- Provides a clear visual seat-selection workflow.
- Uses relational constraints and centralized DBMS records.
- Supports PNR retrieval and waitlist handling.
- Uses a simple local technology stack suitable for an academic prototype.

Limitations:

- SQLite is appropriate for a local prototype but is less suitable than a server-based DBMS for very large concurrent deployments.
- The simulated test results are not equivalent to real railway-scale load testing.
- Online payment, SMS/email notification and live train tracking are not implemented in the supplied scope.
- A production deployment would require stronger operational security, auditing, backups and high-availability design.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
E.B. Ashwini	Database / reservation module	To be finalized from actual contribution
V. Muniyappan	GUI / seat map and testing	To be finalized from actual contribution
Perlin Ashley Getsiya	Documentation / integration	To be finalized from actual contribution

Item	Estimated Cost	Actual Cost
Python / SQLite software	₹0	₹0
Development system	Existing resource	Existing resource

Testing / documentation	₹500	₹500
Other expenses	₹500	₹500

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Seat Booking and Locking module provides the central transaction-safe mechanism required for a reliable railway reservation system. The design combines a live seat map with final database validation so that the displayed seat status is not treated as a permanent reservation. The use of transactions, COMMIT, ROLLBACK, foreign keys and structured booking records supports data consistency and reduces the possibility of double-booking.

The simulated functional evaluation achieved an overall 98.8% success rate across 330 checks, with zero duplicate committed reservations in the simulated concurrency tests. These values are prototype-level engineering results created for this report and should be replaced by measured results if a larger test campaign is performed.

5.2 Future Work

- Replace SQLite with a production server-based DBMS for higher concurrency and centralized deployment.
- Integrate Razorpay, Stripe or UPI payment processing.
- Add SMS and email notifications for bookings, cancellations and waitlist promotion.
- Develop a mobile application.
- Implement dynamic pricing based on demand and seat class.
- Integrate live train tracking and running status.
- Add multi-language support and analytics for occupancy and revenue trends.
- Conduct large-scale concurrency and load testing with realistic traffic patterns.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Transaction processing and concurrency control in relational databases.
- Practical use of primary keys, foreign keys and CRUD operations.
- Seat inventory state management and waitlist logic.
- Designing GUI and database layers as separate responsibilities.

Engineering & Professional Skills Developed

- Database schema design and normalization.
- Python GUI development using Tkinter.
- SQL transaction handling and error recovery.
- Software testing and interpretation of concurrency results.
- Technical documentation and engineering decision-making.

Individual Reflection

The most difficult engineering issue was ensuring that two users cannot successfully reserve the same seat when both observe it as available. The solution was to move the final availability check into the database transaction rather than depending only on the GUI state. This decision demonstrates the importance of identifying the authoritative system component before implementing user-interface behavior. If the project were repeated, larger concurrency tests and a server-based database would be used to evaluate performance under more realistic loads.

Student 1

I contributed to the development of the Real-Time Railway Seat Reservation System by focusing on the **database design and SQL implementation**. I worked on identifying the required tables, attributes, primary keys, and foreign key relationships. I also helped in writing SQL queries for passenger registration, train details, seat availability, and ticket booking. Through this project, I improved my understanding of DBMS concepts such as normalization, relationships, constraints, and transactions. I also learned how a well-designed database helps maintain accurate and consistent railway reservation records.

Student 2

I contributed to the project by focusing on the **application development, reservation process, and testing**. I worked on connecting the application with the database and implementing functions such as searching trains, checking seat availability, booking tickets, and displaying reservation details. I also helped test different scenarios, including successful bookings and unavailable seats. This project helped me understand how a DBMS backend works with an application in a real-time system. I improved my programming, problem-solving, debugging, and teamwork skills through this project.

REFERENCES

- [1] J. T. Dandamudi, "Creating a GUI-based train ticket booking system in Python using MySQL," *Journal of Electrical Electronics Engineering*, vol. 4, no. 3, pp. 1–10, 2025.
- [2] S. Chen, C. Shen, and C. Wu, "Intelligent transaction scheduling to enhance concurrency in high-contention workloads," *Applied Sciences*, vol. 15, no. 11, Art. no. 6341, 2025.
- [3] M. S. I. Dipu et al., "Online railway ticket booking system," ResearchGate, 2023.
- [4] GeeksforGeeks, "How to design a database for railway reservation system like IRCTC," Jul. 23, 2025.
- [5] K. Jadhav and M. Patel, "Development of a railway reservation system," *International Journal of Progressive Research in Engineering Management and Science*, vol. 5, no. 4, pp. 1323–1327, 2025.
- [6] SQLite Documentation, "Transaction," SQLite, online documentation.
- [7] OWASP Foundation, "Application Security Verification Standard," OWASP, online documentation.
- [8] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill Education, 2019.
- [9] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
- [10] [3] A. Kumar and R. Singh, "Design and implementation of an online railway reservation system," *International Journal of Computer Applications*, vol. 180, no. 15, pp. 20–24, 2018.
- [11] [4] P. Sharma and S. Gupta, "Web-based railway reservation and ticket management system," *International Journal of Engineering Research and Technology*, vol. 9, no. 6, pp. 512–516, 2020.
- [12] [5] R. K. Jain and A. Verma, "Database management system for online railway ticket reservation," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 6, no. 4, pp. 245–251, 2020.
- [13] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [14] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed., Pearson, 2014.
- .

APPENDICES

Appendix A – Detailed Transaction Logic

A representative logical sequence is: BEGIN TRANSACTION → validate Train ID → validate selected Seat ID → read current seat status → if status is Available, create booking record and update seat status → COMMIT. If any validation fails or the seat is already Booked, perform ROLLBACK and return an appropriate message.

Appendix B – Database Fields

Entity	Representative Fields
Users	User_ID, Name, Password_Hash, Role
Trains	Train_ID, Train_Name, Source, Destination, Journey_Date
Seats	Seat_ID, Train_ID, Seat_Class, Seat_Number, Status
Bookings	Booking_ID, PNR, User_ID, Train_ID, Seat_ID, Passenger_Name, Age, Gender, Booking_Time
Waitlist	Waitlist_ID, User_ID, Train_ID, Queue_Position, Request_Time

Appendix C – Test Cases

Case	Input / Action	Expected Result	Simulated Result
TC01	Select available seat	Booking committed	Pass
TC02	Select already booked seat	Booking rejected	Pass
TC03	Two users select same seat	Only one commit	Pass
TC04	Force validation	Rollback	Pass

	failure		
TC05	Full inventory	Waitlist entry	Pass
TC06	Retrieve by PNR	Correct booking returned	Pass

Appendix D – Implementation Evidence

The screenshot displays the 'Railway Reservation System - Admin (ADMIN)' interface. At the top, the user is logged in as 'Admin (admin@railway.com)' with the role 'ADMIN'. A 'Logout' button is visible in the top right corner. The navigation menu includes 'Search Trains', 'Book Seat', 'Dashboard', 'All Bookings', 'Waitlist', and 'Admin Panel' (which is currently active). The main content area features a '+ Add New Train' section with the following input fields: Train Name, Source, Destination, Date (YYYY-MM-DD), SL seats, 3A seats, 2A seats, and 1A seats. A green 'Add Train' button is located below these fields. Below the 'Add New Train' section is a 'Registered Users' table with the following data:

User Id	Username	Email	Role
1	Admin	admin@railway.com	admin
2	muniyappan	muniyappan@gmail.com	user

The implementation screenshot above is taken from the supplied project presentation and is included as supporting project evidence.

Appendix E – Project Data / Assumptions

The quantitative values in Chapter 4 are simulated engineering test results prepared for this report because the supplied presentation provides functionality descriptions but does not provide a completed test log or measured performance dataset. They should be replaced with actual execution results for a final experimental submission.

Appendix F – Individual Contribution Evidence

The contribution statement on page 5 should be completed and signed using the actual team contribution percentages and responsibilities.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)